

A. Don't.

Don't Copy Mutable Values Without `copy.copy()` and `copy.deepcopy()`

It's better to think of variables as labels or name tags that refer to objects rather than as boxes that contain objects. This mental model is especially useful when it comes to modifying *mutable* objects: objects such as lists, dictionaries, and sets whose value can mutate (that is, change). A common gotcha occurs when copying one variable that refers to a mutable object to another variable and thinking that the actual object is being copied. In Python, assignment statements never copy objects; they only copy the references to an object. (Python developer Ned Batchelder has a great PyCon 2015 talk on this idea titled, “Facts and Myths about Python Names and Values.” Watch it at <https://youtu.be/AEJHKGk9ns>.)

For example, enter the following code into the interactive shell, and note that even though we change the `spam` variable only, the `cheese` variable changes as well:

```
>>> spam = ['cat', 'dog', 'eel']
>>> cheese = spam
>>> spam
['cat', 'dog', 'eel']
>>> cheese
['cat', 'dog', 'eel']
>>> spam[2] = 'MOOSE'
>>> spam
['cat', 'dog', 'MOOSE']
>>> cheese
['cat', 'dog', 'MOOSE']
>>> id(cheese), id(spam)
2356896337288, 2356896337288
```

A visualization of the execution of this code is at <https://autor.com/listcopygotcha1>. If you think that `cheese = spam` copied the list object, you might be surprised that `cheese` seems to have changed even though we only modified `spam`. But assignment statements *never copy objects*, only references to objects. The assignment statement `cheese = spam`